A Universal Sweep-Ratio Ceiling for Activation Patching: Cache-Replay Restoration and Circuit Discovery in Persistent Computational Graphs

Abdallah Khemais
ISITCOM, University of Sousse

August 2026

Abstract

Activation patching localizes which computations in a neural network cause its behaviour, but a real study sweeps K sites, and tape-based frameworks pay a full forward pass per site. On NeuroDSL's persistent computational graph, a single patch already costs only its downstream subgraph; we eliminate a second, avoidable cost, restoring the corrupted baseline between sites, via cache replay instead of recomputation. Does patching become cheap enough with depth that an exhaustive sweep scales into a fast alternative to approximate methods such as AtP*? We answer with a theorem: the aggregate speedup of an exhaustive per-node sweep is sandwiched by exact bounds, holding at every finite depth for every cost-by-depth profile, converging to exactly 2× whenever no layer asymptotically dominates the network's cost, strengthening a companion study's depth-uniform-only result; when that fails, the bounds converge to an exactly computable 8/3. This ceiling rules out scale or architecture alone as a route to closing the ACDC/AtP* fidelity-cost gap.

We confirm both halves empirically: cache-replay restoration cuts sweep cost by 36.1% on an 8-layer CPU model, rising to 37–39% at GPU scale (up to ∼1.82B parameters) once two confounds are diagnosed and removed. Cross-validated against PyTorch on identical weights (parity 1.23×10^{-6}), the single-patch advantage is a depth-dependent crossover matching the theorem's prediction. Applied to a model trained on the induction task, the same search recovers attention heads reaching 99.94% of the causal effect. Finally, a grafted layer reaches lower loss than an equal-budget control in a single run (an observation, not a seeded claim, since a three-seed test here shows such comparisons can invert sign), while three pre-registered criteria for causal-diagnosis-directed graft placement all fail.

1 Introduction

Mechanistic interpretability seeks to explain a neural network's behaviour by identifying which internal computations are causally responsible for it. **Activation patching** (also called causal tracing) [6, 7] is the workhorse technique for this: run the model once on a "clean" input and once on a "corrupted" input, then copy a single internal activation from the clean run into the corrupted run and observe how much of the output moves back towards the clean behaviour. The unit of a real study is not one patch but a **sweep**: repeating this at every layer (and often every attention head or token position) to localize where the causally relevant computation lives. Tooling such as TransformerLens [8] has made this sweep protocol standard, but in tape-based frameworks its cost is K full forward passes: PyTorch [4] and JAX [5] rebuild the computational graph from nothing on every call, so patching one node costs exactly the same as patching any other, regardless of how much computation actually lies downstream of it.

NeuroDSL [1] maintains a persistent, mutable computational graph in which every node tracks whether its cached value is still valid. Writing a node invalidates only its true downstream dependents (its impact subgraph), and the next demand-driven evaluation recomputes exactly

that subgraph – a mechanism proven to cost $\mathcal{O}(|\mathcal{V}_\theta|)$, independent of total graph size [1]. Activation patching is mechanically the same operation (a node write followed by a demand), so a single patch inherits this bound for free. A full sweep, however, is not just K independent cheap patches: each site must be tested from a common corrupted baseline, so the graph must be restored between sites, and if restoration is done the same way a patch is applied it pays exactly the same recomputation cost as the patch it is undoing – eating back a substantial fraction of the saving a single cheap patch promised. We show this is avoidable: the corrupted baseline was already computed once, in full, before the first patch, so restoring to it requires no computation, only a direct copy of values already sitting in a cache.

This raises a question sharper than an engineering optimization: if per-site cost genuinely shrinks with depth, does an exhaustive, ACDC-style sweep – testing every candidate site, the discipline that makes ACDC [12] exact where fast-but-approximate attribution methods such as AtP* [13] are not – become cheap enough *in aggregate* to close this fidelity/cost gap? We answer this with a theorem before describing any implementation, since the answer depends only on the graph's structure, not on any particular code (Section 4), and then confirm both the theorem and the mechanism that realizes it empirically, from an 8-layer CPU model up to a 1.82-billion-parameter model on GPU, on random weights and on a model trained to solve a task with a documented ground-truth circuit (Section 7).

Contributions.

1. **A universal sweep-ratio theorem.** The aggregate speedup of an *exhaustive per-node* sweep is bounded by an exact pair-counting identity holding at every finite depth for *every* cost-by-depth profile, with no asymptotic hypothesis, and these bounds converge to *exactly* $2\times$ as depth grows whenever no single layer asymptotically dominates the network's cost (Theorem 2, §4) – strictly strengthening a companion study's depth-uniform-only result [3] – together with a matching necessity remark showing that when this hypothesis fails, a single asymptotically dominating layer breaks the ceiling and the ratio converges instead to an exactly computable 8/3 (Remark 2).

2. **Amortized restoration.** The patching mechanism developed in §5 restores a patched site's downstream cone by direct cache replay instead of recomputation – a capability specific to a persistent graph, where every node is individually addressable with an explicit validity state, and one a framework that rebuilds its graph from scratch has no equivalent for.

3. **Composable, searchable patching.** The same mechanism extends to simultaneous multi-site patches with cost bounded by the union of their cones, and to an automated greedy search built on top of it, correctly falling back to recomputation whenever a candidate's cone overlaps an already-selected site's (§5.3–§5.4).

4. **Correctness-first empirical validation** on an 8-layer CPU Transformer (§7): five exact-equality invariants confirmed before any timing claim, a 36.1% reduction in total sweep cost, and a structural-signal analysis showing downstream cone size predicts cost and causal importance across depths but carries no information among same-depth sibling sites.

5. **Cross-framework and GPU-scale confirmation.** Weights exported to a PyTorch re-implementation confirm the theorem's per-site crossover at parity 1.23×10^{-6} (§7.5); at GPU scale (up to ∼1.82B parameters), diagnosing and removing two independent confounds – a GPU clock state and an $\mathcal{O}(|\text{cone}| \times |\text{rules}|)$ invalidation-bookkeeping cost – reverses an initial shortfall into a gain of 37–39%, above the CPU figure, and retracts a roofline-style explanation offered for that shortfall (§7.7).

6. **Validated circuit discovery.** Trained on the induction task [9], the unmodified search recovers a set of attention heads reaching 99.94% of the causal effect; its first two heads are independently confirmed by attention patterns the search never examined, and a backward-pruning pass reduces the set to four layer-1 heads with no measurable loss (§7.8).

7. **Structural surgery.** A structural-mutation primitive extends the same persistence to mutating structure: a layer grafted mid-training into a real trained model reaches a lower validation loss than an equal-budget fresh control, while three pre-registered criteria for letting causal diagnosis *direct* a graft's placement all fail – a clean negative result the cheap diagnose-graft-rediagnose cycle made testable in minutes (§7.9).

A companion paper exploits the same structural-mutation primitive for a third purpose beyond patching and directed capacity placement – growing a network's depth during training – and is reported separately, since it shares only the mechanism with this paper, not its method or question.

All experiments use NeuroDSL's existing, already-tested Llama-style transformer implementation; no new model architecture was written for this paper. Code, tests, and the executed notebooks reproducing every number reported here are available at `https://github.com/nevermind78/NeuroDSL`.

2 Related Work

Activation patching and circuit discovery. Causal tracing [6] and causal mediation analysis [7] localize a behaviour by overwriting a single activation and reading the resulting change; ACDC [12] turns this into an exhaustive search over candidate edges – exact, but paying a full recomputation per site in a tape-based implementation. Tooling such as TransformerLens [8] makes the sweep protocol practical without changing its asymptotic cost.

Approximate attribution. AtP/AtP* [13] estimate every site's effect from a single backward pass, trading exactness for a far larger constant-factor speedup than any locality argument alone can offer, at the cost of well-characterized false-negative modes AtP* is explicitly designed to bound. This paper does not compete with that line: Theorem 2 (§4) instead characterizes exactly where an exhaustive, exact mechanism like the one here helps AtP*'s own recommended pipeline – its exact-verification step over a ranked top-k of deep, late candidates.

Persistent and reactive graph engines. Unlike PyTorch [4] and JAX [5], which rebuild their computational graph from nothing on every call, NeuroDSL [1] keeps a persistent DAG whose invalidation is provably confined to a mutated node's downstream cone [2]. A companion cost-accounting study [3] derives the aggregate cost of a *depth-stratified* sweep under this model, converging to $(q{+}2)/(q{+}1)$ or $q{+}2$ in the Karamata index q of the network's cost profile; Section 4 strengthens this to a single, profile-independent constant under the sampling protocol a real exhaustive sweep or automated search actually uses.

Greedy set search. The greedy search of §5.4 superficially resembles submodular maximization, where greedy addition carries a $1 - 1/e$ optimality guarantee [10]; §5.4 explains why that guarantee does not transfer here, since it constrains the cost of combining sites, not the causal-recovery function actually being maximized.

3 Background: Persistent Graphs and the Impact Subgraph

We rely directly on the persistent-graph model of [1] and only restate what is needed here. The graph stores named tensors and transformation rules that together define a persistent DAG.

Writing a new value to a node via `set!` triggers a downstream invalidation traversal that clears the `valid` flag on the written node and on every node reachable from it through the rule graph. A subsequent `demand!` call walks the (cached) topological order and recomputes only the nodes it finds invalid, leaving everything else untouched.

[1] introduces this engine and proves the analogous bound for the upstream (parameter-update) traversal; the downstream case this paper relies on – that the invalidation routine visits exactly the **impact subgraph** $\mathcal{G}_\theta = (\mathcal{V}_\theta, \mathcal{E}_\theta)$ of the written node θ, the set of nodes reachable from θ in the DAG, giving a recomputation cost of $\mathcal{O}(|\mathcal{V}_\theta|+|\mathcal{E}_\theta|)$ independent of total graph size – is proved by exact double inclusion (soundness and completeness of the reachability traversal) in [2]. This bound was established and validated for *parameter* updates during training. Activation patching is, mechanically, also a node write followed by a demand – so the same bound applies directly to a patch, with no new theorem required, and it is this purely structural fact – cost depends on which nodes lie downstream, not on any tensor value – that Section 4 pushes to its aggregate limit before any mechanism or measurement is needed to answer the question it asks.

4 A Universal Ceiling on Exhaustive Sweep Cost

Because a single patch's cost is a purely structural quantity (Section 3), a sharper question can be asked before any mechanism is described or any number measured: if a search or a systematic study patches *every* candidate site of a network in turn, restoring a shared corrupted baseline between sites, does the total cost of that exhaustive sweep converge to some ceiling as the network grows deeper, and if so, to what? We answer this in full generality here; §7.5 and §7 confirm both the per-site and the aggregate answer empirically, on real reactive graphs and against a real PyTorch implementation.

Definition 1 (Depth-stratified sweep, recalled from [3])**.** *A layered DAG of depth L assigns to layer $j \in \{1, \dots, L\}$ a computational weight $w_j > 0$ (its node count), with $N(L) = \sum_j w_j$, and a site's downstream cone coincides, up to an $O(1)$ per-site remainder that vanishes in the ratios below, with the suffix $\sum_{j>i} w_j$ of the layer i it belongs to. The depth-stratified sweep places S candidate sites per layer, independent of w_j, and its aggregate ratio is $\rho(L) = SL \cdot N(L) \,/\, S \sum_{i=1}^{L} \sum_{j>i} w_j$.*

Theorem 1 (Aggregate sweep ratio, recalled from [3])**.** *If $w_j \sim \ell(j) j^q$ for ℓ slowly varying and $q \geq 0$ (Karamata), then $\rho(L) \to (q+2)/(q+1)$ for this output-heavy profile, and $\to q+2$ for the reversed (input-heavy) profile $w_j \sim \ell(L{+}1{-}j)(L{+}1{-}j)^q$; both recover 2 at $q = 0$. A wall-clock refinement of the same result predicts a ceiling strictly below the combinatorial limit whenever per-site interpreter overhead is a material fraction of per-node cost – already non-binding, by direct measurement, at transformer scale [3].*

We take this statement, and its proof by Karamata's theorem for regularly varying sequences, as given; it is not re-derived here. What it leaves open is whether the q-dependence – and the resulting anywhere-from-1-to-unbounded range of possible ceilings – is a genuine structural property of reactive invalidation, or an artifact of the one sampling protocol Definition 1 formalizes: one fixed representative site per layer, independent of the layer's own size. The sweep this paper actually runs, and the sweep an automated circuit-discovery search or a neuron-/edge-level ablation study runs in practice, tests *every* individual candidate site – so a layer with more attention heads, or a wider MLP, is swept with proportionally more sites, not with the same fixed count as a narrower layer. We show that under this second, equally real protocol, the q-dependence vanishes entirely.

We reuse, without re-proving, the structural locality theorem of [2] recalled in Section 3: reactive invalidation seeded at a mutated node s marks invalid exactly the downstream cone $\mathcal{V}_s^+ = \{v \in \mathcal{V} \mid \text{a path of length} \geq 1 \text{ from } s \text{ to } v\}$, at cost $O(|\mathcal{V}_s^+| + |\mathcal{E}_s^+|)$.

Definition 2 (Exhaustive per-node sweep)**.** *In the layered cost model of Definition 1, assume the weights w_j are integers and layer j consists of w_j unit-cost nodes. The* exhaustive per-node sweep *takes every node as a candidate site, so layer j hosts w_j sites – in contrast with the depth-uniform placement of Definition 1. With $\mathcal{V}_v^+$ the downstream cone of node v (every node of every deeper layer, together with v's descendants inside its own layer, up to the same vanishing per-site remainder as above), write*

$$D(L) \;=\; \sum_{v} \big|\mathcal{V}_v^+\big|, \qquad \rho_{\text{node}}(L) \;=\; \frac{N(L)^2}{D(L)}$$

for the ratio of $N(L)$ independent full recomputations to the cost of exhaustively patching every node, each restored via the structural locality theorem's exact cone.

Theorem 2 (Universality of the exhaustive per-node ratio)**.** *Let $\|w\|_2^2 = \sum_{j=1}^{L} w_j^2$ and $N = N(L)$. For* every *weight profile (w_j):*

(i) (Exact sandwich.) *$D(L)$ counts exactly the ordered comparable node pairs, and*

$$\frac{N^2 - \|w\|_2^2}{2} \;\le\; D(L) \;\le\; \binom{N}{2}, \qquad \textit{hence} \qquad \frac{2N}{N-1} \;\le\; \rho_{\text{node}}(L) \;\le\; \frac{2}{1 - \|w\|_2^2/N^2},$$

the left bound on D (right bound on ρ_{node}) attained iff every layer is an antichain (no two nodes of the same layer comparable), and the right bound on D iff every layer is internally totally ordered – in which case the comparability relation among nodes is a single total order (as realized, e.g., by a fully sequential chain) and $\rho_{\text{node}}(L) = 2N/(N-1)$ exactly, at every finite L, for every profile, with no asymptotic hypothesis whatsoever.

(ii) (Universal limit.) *If $\max_j w_j/N(L) \to 0$ – no single layer carries an asymptotically positive fraction of the total weight – then $\rho_{\text{node}}(L) \to 2$ for every profile and either orientation. This hypothesis is strictly weaker than, and implied by, Theorem 1's Karamata hypothesis: under either profile there, $\|w\|_2^2/N^2 \sim (q+1)^2/\big((2q+1)L\big) \to 0$ for every $q \ge 0$.*

(iii) (Rate.) *In the totally ordered case, $\rho_{\text{node}} = 2 + 2/(N-1)$ exactly, i.e. $\Theta(1/N)$ – which, under a Karamata profile, is $\Theta(1/L^{q+1})$ since $N \sim Lw_L/(q+1)$, strictly faster than $1/L$ whenever $q > 0$. In the layerwise-antichain case, $\rho_{\text{node}} = 2 + \frac{2(q+1)^2}{(2q+1)L}\,(1+o(1))$ under either Karamata profile of Theorem 1: here the rate is genuinely $\Theta(1/L)$, governed by depth alone, since intra-layer width does not shrink the cone in this construction. The two cases coincide only at $q = 0$, where $N \sim L$.*

(iv) (Orientation invariance.) *At every finite L, both bounds in (i) – and ρ_{node} itself in the antichain and totally ordered cases – are invariant under profile reversal $w_j \mapsto w_{L+1-j}$, unlike the asymmetric pair $(q+2)/(q+1)$ vs. $q+2$ of Theorem 1.*

Proof. $D(L)$ counts ordered pairs (v, u) with $u \in \mathcal{V}_v^+$, i.e. comparable pairs. Decompose by layers. *Inter-layer*: every node of layer i has every node of every layer $j > i$ in its cone, contributing $\sum_{i<j} w_i w_j = \frac{1}{2}(N^2 - \|w\|_2^2)$, by expanding $N^2 = (\sum_j w_j)^2 = \|w\|_2^2 + 2\sum_{i<j} w_i w_j$. *Intra-layer*: layer i contributes its own count D_i of comparable ordered pairs, $0 \le D_i \le \binom{w_i}{2}$, extremes attained by an antichain and a total order respectively. Summing,

$$\frac{N^2 - \|w\|_2^2}{2} \le D(L) \le \frac{N^2 - \|w\|_2^2}{2} + \sum_i \frac{w_i(w_i-1)}{2} = \frac{N^2 - N}{2} = \binom{N}{2},$$

proving (i); the per-site remainders of the layered cost model total $o(N^2)$ and vanish in the ratio. For (ii), $\|w\|_2^2/N^2 \le (\max_j w_j/N) \cdot (\sum_j w_j/N) = \max_j w_j/N \to 0$ and $N \ge L \to \infty$;

both bounds of (i) converge to 2, so $\rho_{\text{node}} \to 2$ by squeezing. Under a Karamata profile, w_j^2 is regularly varying of index $2q$, so Karamata's theorem gives $\|w\|_2^2 \sim L w_L^2/(2q+1)$, and with $N \sim L w_L/(q+1)$ (as in Theorem 1's proof) the ratio is $\sim (q+1)^2/((2q+1)L)$; the reversed profile leaves N and $\|w\|_2^2$ unchanged, both being symmetric functions of the multiset $\{w_j\}$. (iii) follows from $2/(1-x) = 2 + 2x + O(x^2)$ at $x = \|w\|_2^2/N^2$, and from (i) in the totally ordered case. (iv) is immediate: N and $\|w\|_2^2$ are invariant under $j \mapsto L+1-j$, and in the two extreme cases ρ_{node} is a function of these two quantities alone. □

Remark 1 (Sampling design, not graph structure)**.** *Theorem 1 and Theorem 2 are a matched pair: any aggregate ratio of this kind equals* $1/\mathbb{E}[U]$ *for* U *the downstream-weight fraction of a site drawn uniformly* among the sites*, and the two theorems differ only in the law this induces on depth. Depth-uniform placement draws depth uniformly, and* $\mathbb{E}[U]$ *reads the cost profile directly – the q-dependent limits of Theorem 1. Per-node placement draws depth* size-biased *by weight, and then, writing* p *for the (continuum limit of the) normalized weight density and* $\bar{P}(t) = \int_t^1 p$ *its survival function,* $\mathbb{E}[U] = \int_0^1 p(t)\bar{P}(t)\,dt = \frac{1}{2}[\bar{P}(0)^2 - \bar{P}(1)^2] = \frac{1}{2}$ *identically, for every density* p*; discretely, this continuum identity is nothing but the pair-counting identity* $\sum_{i<j} w_i w_j = (N^2 - \|w\|_2^2)/2$ *used in the proof above, an algebraic fact that requires no regular variation and holds at every finite* L*. Neither regime is an artifact of the other: standard activation-patching protocols that probe one representative site per layer are the depth-stratified regime of Theorem 1, while an automated circuit-discovery search or a neuron-/edge-level ablation study that does not restrict itself to one site per depth – exactly what the sweep procedure and the greedy search run when their candidate pool is every head or every neuron of every layer – is the per-node regime of Theorem 2. The universal constant* 2 *is the signature of size-biased (genuinely exhaustive) placement, approached from above at a rate governed by depth (*$\Theta(1/L)$*) when layers have genuine width, or faster (*$\Theta(1/N)$*) on a fully sequential chain (Theorem 2(iii)). A further, immediate corollary of the same closed form: the ratio for a single site* individually*,* $N/|\mathcal{V}_v^+|$*, grows without bound as its relative depth approaches* 1 *(the deepest sites), even though the sweep's aggregate stays pinned near* 2 *– exactly the regime Section 7 confirms both empirically (§7.5's PyTorch crossover) and in aggregate (§7's GPT-2-scale measurement).*

Remark 2 (The no-macroscopic-layer hypothesis is necessary)**.** *Hypothesis (ii) cannot be dropped when layers have genuine width: if a single antichain layer carries* $w_{j_0} = \lceil N/2 \rceil$ *and the rest of the weight is spread thinly across the remaining layers,* $\|w\|_2^2/N^2 \to \frac{1}{4}$ *and* $\rho_{\text{node}}(L) \to 8/3 > 2$*: the sites of the heavy layer are mutually invisible (none lies in another's cone, being an antichain), which deletes a positive fraction of the comparable pairs from* $D(L)$*. If instead the heavy layer – and every other layer – is internally totally ordered,* $\rho_{\text{node}} = 2N/(N-1)$ *regardless of the heavy layer's size, by part (i)'s exact identity: universality is unconditional only up to intra-layer order, not layer size alone. §7 validates this necessity directly, at zero numerical tolerance, on a real reactive graph built specifically to violate hypothesis (ii).*

5 Mechanism: Amortized Restoration

The theorem above is a statement about graph structure alone; realizing its saving in practice requires a mechanism, described here at the level of precision needed to state and check its correctness and cost.

5.1 Value Injection with Targeted Invalidation

`set!` is designed to write leaf values – inputs and parameters, nodes with no incoming rule. Calling it on such a node clears the node's own `valid` flag, which is harmless there, since `demand!` skips recomputing a node with no rule regardless of that flag. Patching targets an

internal node instead – one that already has a rule attached (e.g. a transformer block's residual-stream output) – for which clearing the node's own flag is not harmless: the next `demand!` would recompute it from its rule, using its real (still corrupted) inputs, and the injected value would be lost before it could be observed. The patch primitive avoids this with no new graph-level mechanism, only a different composition of the existing ones: it mutates the target node's value and validity state directly, so the node itself is never recomputed, and invalidates only its true consumers by calling the same downstream invalidation traversal (Section 3) on each of them individually, using the cached consumers index of §7.7.1 rather than scanning every rule. One correctness detail is load-bearing, not defensive style: the value written must be an explicit copy, not an alias – on a device where the cache value is already a resident array, aliasing the node's stored value directly onto the cache's own buffer would let a later recomputation overwrite the cache silently. This was caught empirically, not by inspection: a search's own trajectory reported real recovery while an independent re-check of the identical patch set reported none, traced to exactly this aliasing. Two further routines complete the single-patch interface: one copies every node's value in a namespace into a plain dictionary (the copy is necessary since a live value is an alias into the graph's own buffer), and a recovery metric computes the standard causal-tracing normalization, $1 - \|\text{patched} - \text{clean}\| / \|\text{corrupted} - \text{clean}\|$.

5.2 Restoration Without Recomputation

A sweep tests K sites against the same corrupted baseline, so each site's test must end with the graph restored to that baseline before the next site is tested. Restoring the same way a patch is applied – writing the corrupted value back and letting the graph re-invalidate and recompute – pays exactly the same recomputation cost as the patch it undoes. But the corrupted run was computed once, in full, before the first patch, and its entire state already sits in a captured dictionary of node values: restoring a patched site's downstream cone to that baseline is therefore not a computation to redo, but a lookup already answered. The cache-replay restoration primitive writes the cached values directly into each node of the cone and marks it valid, executing no rule; a read-only helper computes that cone with exactly the traversal the invalidation routine would use, so the set of nodes restored is, by construction, identical to the set a patch would have invalidated – the fact that makes the correctness argument of §7.1 an equality, not an approximation. This capability is specific to a persistent graph: a framework that rebuilds its computation from scratch on every call has no individually addressable node to write a cached value into, and no primitive corresponding to "restore this, without recomputing it." The sweep procedure orchestrates the full loop – patch, measure, restore – for a list of sites, changing only the restoration step relative to a naive implementation; the measurement step, the actual scientific quantity of interest, is untouched.

5.3 Composable Multi-Site Patching

Path patching and its variants need to patch several nodes at once and observe their joint effect. The invalidation mechanism underlying the patch primitive never assumed a single node changes at a time: patching several nodes and calling `demand!` once already computes the union of their downstream cones correctly, with any shared descendant recomputed only once. A multi-site patch routine exposes this with no new graph-level primitive, simply applying the single-site patch to each site in turn before the shared `demand!` call. For two sites where neither is reachable from the other (e.g. two attention heads merged only downstream by a later concatenation), applying them in either order gives identical results, since neither patch's invalidation can reach the other's node – the commuting case §7 evaluates. For two sites in an ancestor-descendant relationship, the multi-site patch applies patches in list order, with the last write to a shared location winning – the same well-defined semantics as any sequential in-place update.

5.4 Automated Search and Its Limits

A combined patch of m sites costs at most the sum of their individual costs, and typically less when their cones overlap – a coverage function, the canonical example of a submodular set function. This makes it practical to search over combinations rather than rely solely on independent single-site attributions: the greedy search procedure iteratively adds, from a candidate pool, whichever remaining site most increases joint recovery, stopping once no candidate improves on the current best. It is tempting to read the submodularity of the cost side as licensing the classical $1 - 1/e$ greedy guarantee for submodular maximization [10], but that guarantee applies to the function being *maximized* – here, recovery, not cost – and nothing in this paper establishes that recovery is submodular in the set of patched sites (it is a property of the network's nonlinear response, not a counting function over reachable nodes; §7 shows it is not even additive). The search is therefore an adaptive heuristic with no proven approximation guarantee, validated empirically instead (by independent recomputation, Invariant 5 of §7.1), despite superficially resembling a setting where a guarantee would exist.

Each step of the search tests a candidate *on top of* whatever sites are already selected and still active – different from every use of cache-replay restoration in §5.2, which always restores from the fully corrupted baseline. If the candidate's cone overlaps a selected site's cone (unavoidable once two sites share a downstream merge point, §5.3), restoring the candidate from the corrupted cache would also reset the shared region to its *fully* corrupted value, silently discarding the selected site's still-active contribution. This is not a defect in cache-replay restoration – its guarantee was always specific to undoing one patch from a fully corrupted baseline, and continues to hold exactly there – but it means a different procedure is needed while a search is in progress. An adaptive-restoration step applies cache-based restoration only where that guarantee holds – comparing the candidate's cone against the union of already-selected cones, a set computation requiring no graph evaluation – and falls back to recomputation-based restoration, which remains correct regardless of overlap, everywhere else.

6 Experimental Setup

Unless stated otherwise, we use NeuroDSL's existing, already-tested Llama-style transformer implementation (8 layers, hidden dimension 128, 8 attention heads, sequence length 16) – no new architecture was written for this paper. The model is fed a random $(16, 128)$ matrix directly as its input node, with no token or positional embeddings and no output head, since demonstrating the systems capability requires no language-modelling semantics.

Corruption protocol. We generate a clean input, then corrupt it by replacing a single token's row (position 1) with fresh noise – the standard single-token causal-tracing protocol.

Patch granularity. Cost is measured by patching each layer's entire output tensor, since cost does not depend on which positions are patched, only on how much computation lies downstream. Recovery is measured by patching only the corrupted token's row: replacing a layer's entire output with its clean counterpart makes every subsequent computation identical to the clean run by determinism, which gives no depth-dependent signal, whereas patching only the affected position reveals where its influence is reintegrated into the network.

7 Experimental Results

Every result below is tied to a specific claim made above: exactness of the mechanism (§7.1), the cone-based cost model of Section 3 (§7.2), the restoration saving of §5.2 (§7.3), the search of §5.4 (§7.3), the per-site crossover and aggregate ceiling of Theorem 2 (§7.5–§7.6), the mechanism's

behaviour at production scale (§7.7), and whether the search finds anything real (§7.8), before finally extending it beyond patching to structural mutation (§7.9).

7.1 Correctness Invariants

Following the same discipline as [1] – correctness before any speed claim – every mechanism above is validated by an invariant that does not depend on comparing two implementations that could share the same flaw, all confirmed before any timing measurement was taken.

- **Invariant 1 (survival).** A node's value immediately after a single-site patch equals the same value after a subsequent `demand!` call on the output. Maximum absolute difference: **0.0** (exact).
- **Invariant 2 (last-layer identity).** In our test model the output node *is* the last block's residual-stream output, so patching one is structurally patching the other: recovery must equal exactly 1.0. Measured: **1.000000**.
- **Invariant 3 (restoration equivalence).** For the same site, patched from the same corrupted baseline, the full graph state after cache-replay restoration matches the state after a patch followed by `demand!` exactly, node for node – the invariant that licenses §7.3: any wall-clock difference between the two paths is pure overhead removed, not a shortcut that changes what is measured.
- **Invariant 4 (commutativity).** For two sibling attention heads (§5.3), the output after a joint two-site patch matches the output after the same two patches applied in reverse order exactly.
- **Invariant 5 (search reproducibility).** After the greedy search returns, patching all selected sites at once on a graph freshly reset to the corrupted baseline – a code path entirely separate from the search's internal bookkeeping – reproduces the search's own final recovery exactly.

7.2 Cost and Recovery vs. Depth

Table 1 reports, for each layer, its downstream cone size $|\mathcal{V}_\theta|$ (computed read-only before patching), the wall-clock cost of patching that layer and recomputing the output (trimmed mean over 15 alternated runs, controlling for CPU clock variance as in [1]), and the recovery achieved by patching only the corrupted token's position. The reference full-forward cost is 6.201 ± 0.302 ms, measured with the same rigor.

Table 1: Patch cost, downstream cone size, and recovery vs. layer depth, 8-layer Llama-style model (dim=128).

Layer	Cone size	Cost (ms)	Cost (% of full forward)	Recovery
1	484	5.418	87.4%	0.753
2	415	4.606	74.3%	0.666
3	346	3.831	61.8%	0.613
4	277	3.071	49.5%	0.565
5	208	2.191	35.3%	0.521
6	139	1.444	23.3%	0.461
7	70	0.734	11.8%	0.435
8	1	0.000	0.0%	0.386

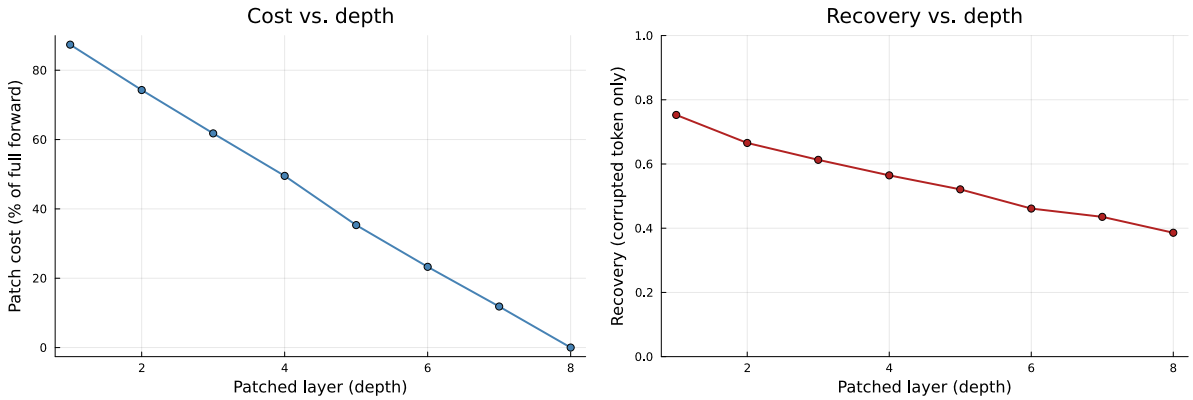

Figure 1: Left: patch cost as a percentage of a full forward pass, by layer depth. Right: recovery of the corrupted token's position, by layer depth. Both decrease monotonically with depth; cost reaches exactly zero at the output layer, where nothing remains downstream to recompute.

Cost decreases monotonically with depth, reaching exactly 0.0% at the output layer, where the impact subgraph is empty by definition – directly confirming Section 3's cone-based cost model.

A cross-check on measurement rigor. The cost column above is cumulative (patching layer i recomputes the entire downstream suffix), so it has no reason to sum to any particular total; the *marginal* cost of layer i is instead the disjoint contribution specific to that layer alone, $\text{marginal}(i) = \text{cumulative}(i{-}1) - \text{cumulative}(i)$. By the discrete analogue of the fundamental theorem of calculus, summing the marginal costs back up must telescope to exactly the full-forward cost, for any monotone nested cost sequence. Table 2 reports this decomposition as an independent cross-check.

Table 2: Marginal cost per layer – disjoint contributions that sum to the full-forward cost.

Layer	Marginal cost (ms)	Marginal cost (% of full forward)
1	0.783	12.6%
2	0.812	13.1%
3	0.775	12.5%
4	0.760	12.3%
5	0.880	14.2%
6	0.746	12.0%
7	0.710	11.4%
8	0.734	11.8%
Sum	**6.200**	**100.0%**

The sum matches the independently-measured full-forward cost (6.201 ms) to within 0.001 ms, consistent with the two quantities being the same underlying cost decomposed two different ways.

7.3 Amortized Sweep, Composition, and Search

Table 3 compares the total cost of a full 8-site sweep under the two restoration strategies of §5.2 (trimmed mean over 10 alternated runs).

Table 3: Total cost of an 8-site sweep, by restoration strategy.

Restoration strategy	**Total sweep cost (ms)**
Recomputation (patch, then re-evaluate)	52.79 ± 1.49
Cache replay (direct restoration, no recomputation)	$\mathbf{33.72 \pm 2.24}$

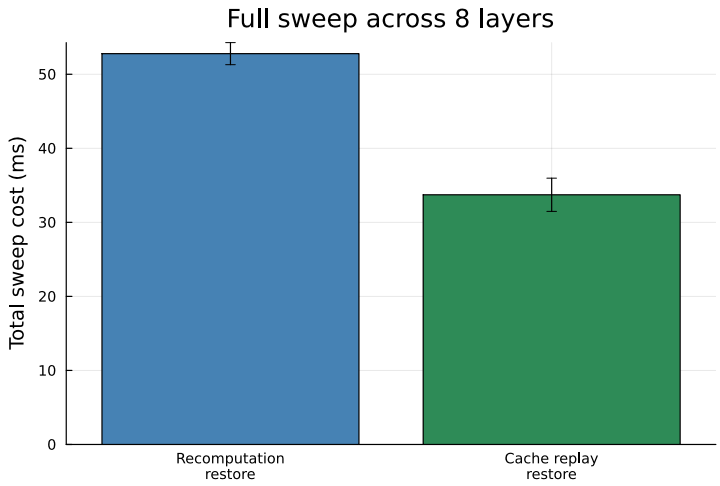

Figure 2: Total cost of a full 8-site sweep, recomputation-based vs. cache-replay restoration.

Cache-replay restoration reduces total sweep cost by **36.1%**, verified by Invariant 3 to leave the graph in an identical state either way – confirming §5.2's claim directly, additively on top of the per-patch savings of Table 1.

Composable patching. We patch two sibling attention heads (heads 2 and 3 of layer 1), individually and jointly. Table 4 shows the joint recovery (0.0647) below the additive sum of the individual recoveries (0.0723), confirming the multi-site patch computes the network's actual (nonlinear) joint response rather than a superposition of independent effects. The two heads' cones overlap almost entirely (493 nodes each, union 494): patching both at once costs essentially half of treating them as unrelated single-site patches.

Table 4: Individual vs. joint recovery for two sibling attention heads.

Patch	**Recovery**
Head 2 alone	0.0295
Head 3 alone	0.0427
Additive sum (head 2 + head 3)	0.0723
Both heads, jointly	**0.0647**

Automated search. Table 5 reports the greedy search's trajectory over the 8 attention heads of a single layer: it stops after 4, none of the remaining heads improving on a cumulative recovery of 0.0977. Recomputing this combination independently as a fresh joint patch on a graph reset to the corrupted baseline gives an identical 0.0977 (Invariant 5) – the adaptive restoration of §5.4, including its overlap-triggered fallbacks, never desynchronized the graph from an independent recomputation.

Table 5: Greedy search trajectory over 8 candidate attention heads (one layer).

Step	Site added	Cumulative recovery
1	Head 3	0.0427
2	Head 5	0.0682
3	Head 2	0.0864
4	Head 8	0.0977

7.4 Structural Signal: Cone Size vs. Causal Importance

Since Table 1 already reports each layer's cone size alongside cost and recovery, a natural question is whether cone size – computable without ever running a patched forward pass – signals causal importance, or only recomputation cost. Across the 8 layers, cone size correlates almost perfectly with cost ($r = 0.9997$, expected: both count the same recomputed node set) and closely with recovery ($r = 0.9928$) – but this second correlation is confounded, since cost and recovery both track depth. Holding depth fixed is more informative: Table 6 repeats the corrupted-token-only recovery protocol on each of layer 1's 8 attention heads individually. Because all 8 heads merge into the same downstream chain, their cones are not merely similar but *identical* (493 nodes each); their individual recoveries, in contrast, vary meaningfully and even in sign (-0.0082 to $+0.0237$).

Table 6: Downstream cone size and individual recovery for 8 sibling attention heads (layer 1, depth held fixed).

Head	Cone size	Recovery
1	493	-0.0071
2	493	$+0.0200$
3	493	$+0.0237$
4	493	-0.0082
5	493	$+0.0198$
6	493	-0.0029
7	493	$+0.0019$
8	493	$+0.0108$

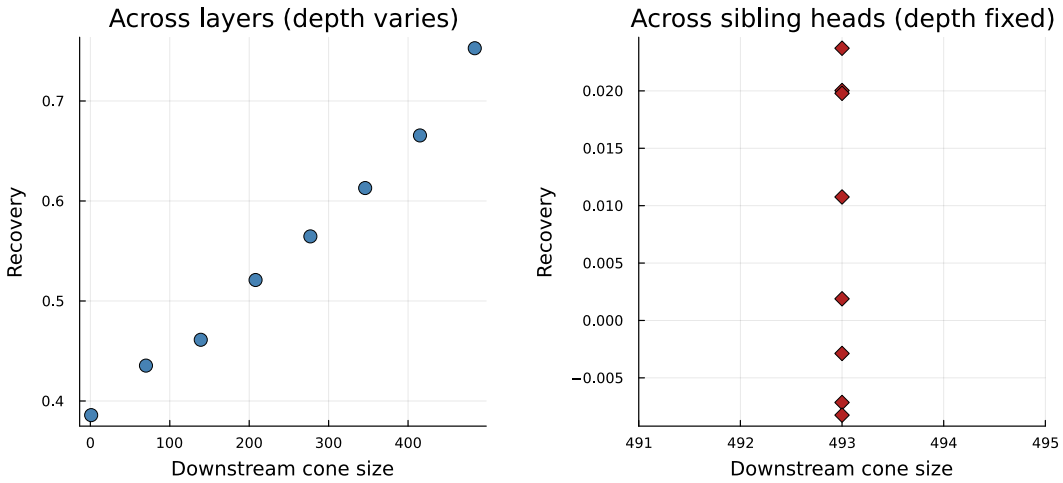

Figure 3: Downstream cone size vs. recovery. Left: across the 8 layers, where depth drives both quantities together. Right: across the 8 sibling heads, where cone size is constant but recovery still varies.

Cone size is therefore a reliable proxy for recomputation cost at any depth, and tracks the trend of causal importance across depths, but among sites sharing a depth it carries no information – distinguishing which one matters still requires the direct measurement this paper's mechanism makes cheap.

A structural caveat under batched attention. NEURODSL also offers an opt-in batched multi-head attention path (off by default everywhere else in this paper), fusing all heads' projections into two shared tensors per layer for throughput. This complicates cone size exactly where Table 6 found it uninformative: patching a query or key site under batching pulls in every sibling head's downstream chain (a cone excess of $4n-2$ for n heads), against an excess of only n past the first fusion point and 0 for the post-softmax-value output site. Both terms were derived from the graph construction and verified against measurement afterward, matching exactly on all three tested depths (Figure 4) – a real, precisely characterized cost that opt-in batching trades against its throughput gain, not a caveat affecting any other measurement in this paper.

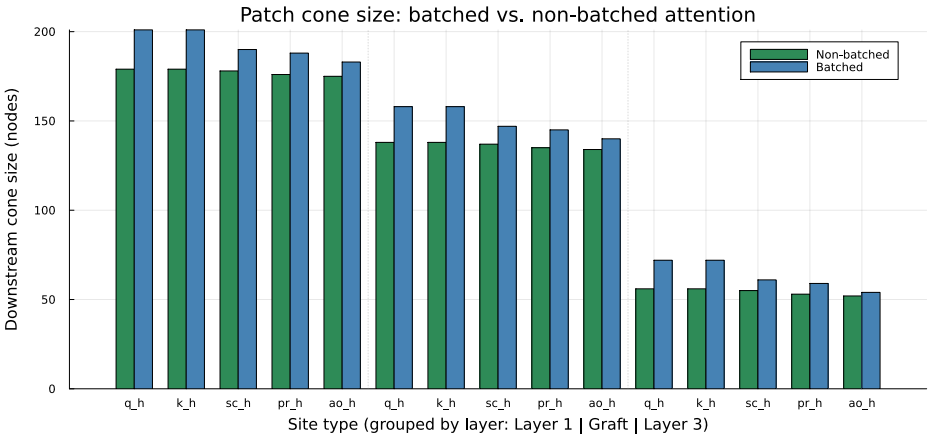

Figure 4: Downstream cone size, batched vs. non-batched attention, three depths. Query/key sites cost far more under batching ($4n-2$) than post-softmax or output sites (n or 0).

7.5 Cross-Framework Validation Against PyTorch

Everything above confirms cost and correctness on NeuroDSL alone; we now benchmark against a real PyTorch run on identical weights, to confirm the per-site crossover Remark 1 predicts. Weights from a 5-effective-layer Llama-style model (4 original layers plus one inserted mid-training by the structural-mutation primitive, §7.9) are exported through NeuroDSL's existing serialization into a PyTorch re-implementation of the same architecture. Parity is verified before any timing claim: forward-pass output matches to a maximum absolute difference of 1.23×10^{-6}. Three methods are then compared across 20 attention-head sites (5 layers × 4 heads): the sweep procedure, unmodified; a naive PyTorch hook recomputing a full forward pass per site; and a hand-optimized PyTorch variant resuming from a cached input to the patched layer but still recomputing an entire layer's attention to patch one head.

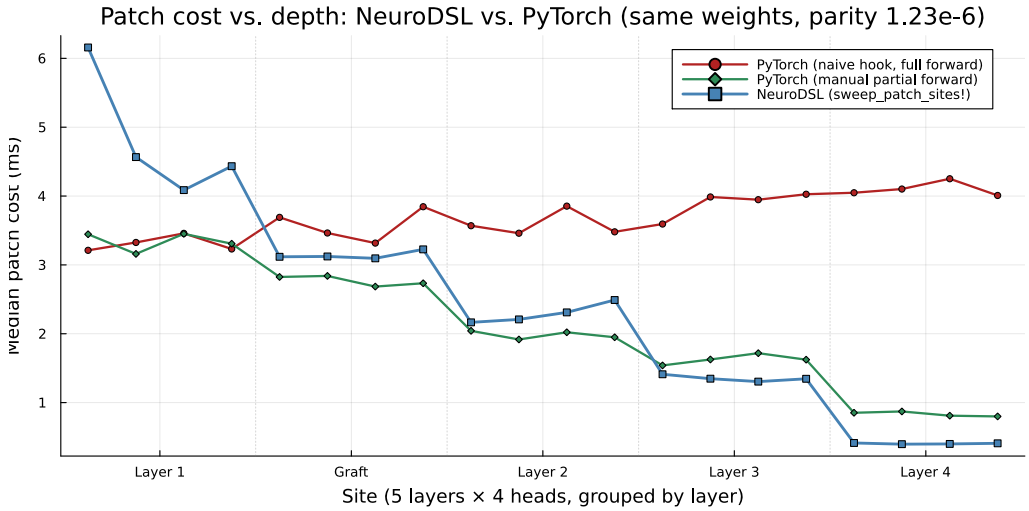

Figure 5: Patch cost vs. depth, identical weights (parity 1.23×10^{-6}), three methods. NeuroDSL trails both PyTorch variants at the shallowest site, where the downstream cone is close to the entire remaining graph, and leads the naive hook by up to 10× at the deepest.

The naive hook's cost is flat across depth, exactly as the structural argument predicts. NeuroDSL is *slower* at the shallowest site, where there is little for it to skip and the constant overhead of many individually-addressable kernel launches costs more than it saves; it overtakes the hand-optimized variant by the third layer and reaches up to 10× the naive hook's cost at the deepest site. This is exactly Remark 1's individual-ratio remark confirmed on real hardware: the advantage is a crossover, not a uniform speedup – a full forward pass costs 6.34 ms on NeuroDSL versus 3.26 ms on PyTorch, the same ∼2× gap the shallowest-site comparison reflects, consistent with NeuroDSL not aiming to out-compute cuBLAS on raw FLOPs.

7.6 Confirming the Universal Ceiling

Section 7.5 confirms a single patch's advantage grows with depth. Since Theorem 2 is a statement about the *aggregate* cost of testing every site, its confirmation needs no trained model or task, only graph structure: on a Llama-style model with 12 layers, 12 heads, dim = 768 (GPT-2-small's shape), batched attention, random weights, we exhaustively sweep the 156 sites ACDC-style coverage would test (144 attention-head outputs, 12 MLP outputs). The aggregate ratio – naive-hook cost divided by the sweep procedure's actual cost – is 2.12×. At roughly double the scale (24 layers, 16 heads, dim = 1024, 408 sites), the ratio is 2.10×: essentially unchanged, not the movement toward 3× the depth-dependent crossover of Section 7.5 alone might suggest. Both

configurations are depth-uniform, the simplest instance of Theorem 2; the exact closed-form cone-size formula reproducing all 564 measured cone sizes with zero residual, and the corresponding $\beta = 0$ predictions (2.145, 2.073) the two measured ratios sit within 1.3% of, are established in full in the companion cost-accounting study [3] and not re-derived here.

Zero-tolerance validation on non-uniform profiles. Both GPT-2-scale architectures above are depth-uniform by construction, so they cannot alone distinguish Theorem 2 from Theorem 1. We validate Theorem 2 directly, at zero numerical tolerance, using the downstream-cone helper on graphs built specifically with non-uniform per-layer weight. On a fully sequential chain ($N \approx 4000$ nodes, three Karamata profiles, $L \in \{20, 80, 320\}$), treating every node as a site gives $D(L) = \binom{N}{2}$ *exactly* in every configuration, hence $\rho_{\text{node}} = 2N/(N-1) \approx 2.0005$ throughout, independent of L at fixed N – confirming the $\Theta(1/N)$ rate of Theorem 2(iii), where Theorem 1 would instead predict, on the very same graphs, values ranging from 1.5 to 4.0. On layered graphs with genuine per-layer weighting ($w_j \propto j^q$ or $(L-j+1)^q$, $L \in \{20, \ldots, 320\}$, $q \in \{0, 0.5, 1, 2\}$), the identity $2\sum_i w_i T_i = N^2 - \|w\|_2^2$ holds exactly at every configuration tested, $\rho_{\text{node}}(L)$ converges to 2 from above at the predicted rate (e.g. $q = 2$: predicted 1.8, measured 1.778 at $L = 320$) *identically* under both orientations – direct confirmation of Theorem 2(iv), in sharp contrast to Theorem 1's asymmetric limits on the same profiles. Finally, a layered graph built to violate hypothesis (ii) directly – one artificially macroscopic layer holding half the total node count – gives $\rho_{\text{node}}(L) \to 8/3 \approx 2.667$, not 2 (measured 2.7701 at $L = 10$, decreasing monotonically to 2.6892 at $L = 80$): Remark 2 confirmed on a real graph, not merely as an algebraic possibility.

What the GPT-2-scale measurement rests on. The two measured ratios (2.1205, 2.1009) sit close to, and are consistent with converging toward, 2 from above, exactly as both Theorem 1 (at its $q = 0$ instance) and Theorem 2 (unconditionally, since hypothesis (ii) holds trivially when every layer carries the same weight) predict. What Theorem 2 adds is not a different number on *this* architecture, but scope: it guarantees the same $2\times$ limit on a Transformer with heterogeneous layer widths, a case Theorem 1 alone would leave open anywhere between 1 and unbounded, and a case the non-uniform checks above validate directly. **An exhaustive sweep of this kind cannot exceed roughly $2\times$ against a naive-hook baseline, at any scale and for any cost-by-depth profile short of a single dominating layer** – not a threshold this mechanism happened to miss twice, but one it cannot cross by construction, on any architecture a real Transformer sweep would present.

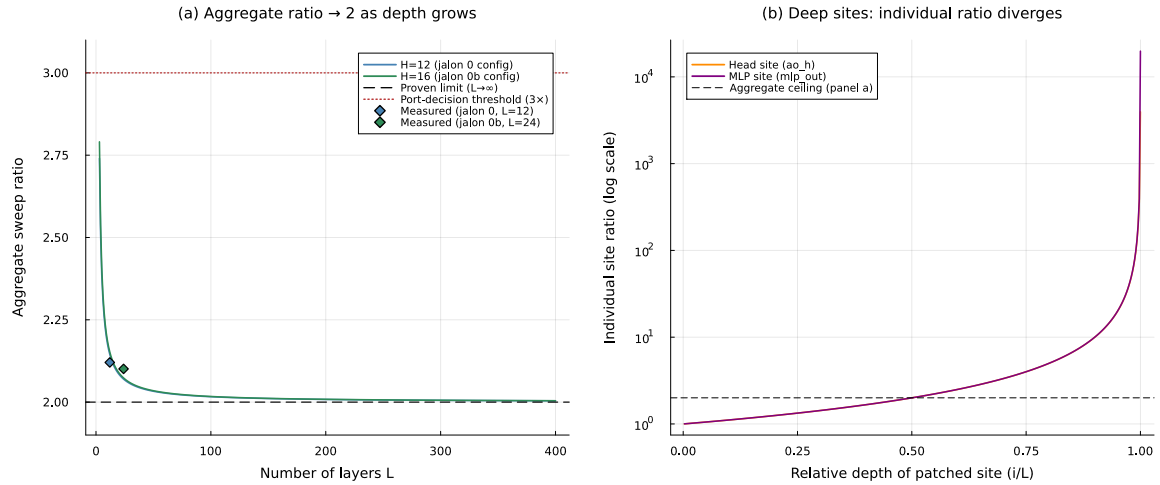

Figure 6: (a) Aggregate sweep ratio vs. depth (exact formula, derived in [3]), with the two measured GPT-2-scale configurations overlaid: both converge toward the universal limit of 2 proved in Theorem 2, both far short of the 3× port-decision threshold. (b) The same closed form evaluated per site ($L = 400$, $H = 12$): the individual ratio grows without bound as relative depth $\to 1$ – the regime AtP*'s own top-k verification step already targets.

Where the mechanism helps. Figure 6(b) confirms Remark 1's individual-ratio remark: unlike the aggregate ratio, the per-site ratio grows without bound as relative depth approaches 1 – exactly the shallowest-cost/deepest-benefit crossover measured in §7.5, and exactly the workflow AtP* itself recommends: rank every site cheaply, then verify only the top-k exactly. NeuroDSL is not positioned to replace an exhaustive ACDC-style sweep with something 3× cheaper; it is positioned to make the exact-verification half of AtP*'s own recommended pipeline nearly free, in exactly the depth regime that pipeline already prefers.

7.7 Confirmation at GPU Scale

Every result above runs on CPU, on an 8-layer, dim-128 model. Recomputation cost depends on graph structure, not weight values, so correctness transfers trivially to any scale – but the *relative size* of the amortization gain is a question of hardware cost structure. We scale the Llama-style model to 16 layers, hidden dimension 2048, 16 heads (∼0.81B parameters) on an NVIDIA RTX A5500 Laptop GPU (17.2 GB VRAM), with no changes to the patching mechanism – every mechanism above is already backend-agnostic. Invariants 1–3 (§7.1) are re-run first and hold exactly on GPU, confirming correctness does not depend on scale or device.

7.7.1 Two Confounds, Found and Removed

Per-layer cost still decreases with depth, but individual measurements are far noisier than on CPU (run-to-run ranges of 3–6×, against a few percent on CPU). Querying the GPU's own monitoring interface during an active benchmark identifies the cause: the GPU runs at 705–795 MHz against a rated boost of 2100 MHz, in a low power state rather than its full boost state, while drawing under half its power budget – not thermal throttling (60–61°C, well under any limit), but the driver's own power-management heuristic declining to boost clocks for a workload of many brief, bursty kernel launches. This is a genuine hardware/driver characteristic, not a correctness problem, but a fluctuating clock confounds any comparison between restoration strategies whose kernel-launch patterns differ. With the machine owner's explicit permission, we lock the core clock to a fixed, moderate 1402 MHz; this does not disable hardware thermal

protection, and observed temperature under sustained load stays at 57–62°C. Locking the clock shrinks run-to-run ranges from 3–6× to 1.1–1.5×, matching CPU-level stability.

A second, independent bottleneck was found by profiling the restoration path itself: cache-replay restoration loops over the cone one node at a time, and at this scale each per-node copy costs $\sim$0.011–0.016 ms regardless of tensor size – the fixed cost of *submitting* an operation to the driver under WDDM, not data volume. A batched variant (new) removes the redundant launches with a single custom kernel assigning one CUDA block per node, restoring an entire cone in one launch; verified byte-identical to the unbatched version before any speed claim, it reduces layer-1 restoration from 20.5 ms to 7.7 ms in isolation, a 2.7× speedup, and remains opt-in (the unbatched version stays the default everywhere).

Separately, a follow-up investigation into CPU-side bookkeeping found that the downstream invalidation traversal (§5.1) scans *every rule in the namespace* to find a node's consumers, once per node visited – an $\mathcal{O}(|\text{cone}| \times |\text{rules}|)$ cost, not the $\mathcal{O}(|\text{cone}| + |\text{edges}|)$ the impact-subgraph bound assumes: measured in isolation on a 9217-node, 6912-rule synthetic graph, a single patch call on a 28-node cone cost 2.373 ms of pure bookkeeping before any recomputation. A cached symbol-to-consumers index (new) replaces the repeated scan with a lookup table, verified against the full 169-test suite and a 300-operation randomized replay producing byte-identical graph state; this reduces the isolated 2.373 ms cost to 0.0056 ms, a 423× reduction, with no growth across the range tested (73 to 9217 nodes). Because every GPU measurement in this section calls the patch primitive at least once per site, this bookkeeping cost was folded into every number measured before this fix, undiagnosed.

7.7.2 The Amortization Gain, Measured Under Controlled Conditions

Table 7 compares the three restoration strategies across two independent launches, clock locked, consumers-index fix in place, each a median over 20 runs.

Table 7: Total cost of a full 16-site sweep at GPU scale ($\sim$0.81B parameters), clock locked to 1402 MHz, by restoration strategy, two independent launches (median over 20 runs each), with the consumers-index fix in place.

Restoration strategy	**Launch 1 (ms)**	**Launch 2 (ms)**
Recomputation (patch, then re-evaluate)	1122.9	1114.2
Cache replay	833.2	818.0
Cache replay, batched	**706.1**	**681.2**

The two launches agree closely: 25.8%/26.6% gain for the existing mechanism, 37.1%/38.9% for the batched one – both substantially higher than the 18.1–20.7%/31.7–32.2% this same protocol measured before the consumers-index fix, since that bookkeeping cost inflated the recomputation baseline (two patch calls per site) more than either restoration strategy (one call per site). The batched kernel now recovers 37–39% of the recomputation cost, *exceeding* the 36.1% measured on CPU (Table 3), reversing rather than merely narrowing the shortfall an earlier pass of this investigation had found.

7.7.3 The Roofline Argument, Retracted

That earlier shortfall (18–21%/31–32% against 36.1%) was originally explained by a roofline-style argument [11]: recomputing a cone is compute-bound, while restoring from cache is memory- or launch-bound, and a GPU's much higher FLOP-per-byte machine balance should make memory-bound restoration relatively more expensive on GPU than on CPU. That argument was internally coherent, but other things were not equal: the invalidation-bookkeeping cost above was paid on both sides of the comparison, but disproportionately inflated the recomputation baseline.

Removing that confound does not merely shrink the shortfall – it reverses it, to a gain *above* the CPU figure. This is worth stating plainly rather than quietly editing away: the shortfall the roofline argument was built to explain was substantially, perhaps entirely, an artifact of an unrelated engineering bug, not a hardware-level effect. A purely theoretical rebuttal would not have found this; only further engineering work on the mechanism itself did. The argument's separate, still-live prediction – that gain should rise with model width d, since recomputation scales as $\mathcal{O}(d^2)$ against restoration's $\mathcal{O}(d)$ – concerns a trend across scale rather than the absolute gap just retracted, and is tested next.

7.7.4 Testing the Prediction Across Scale

We repeat Table 7's measurement at two further widths, keeping head dimension and the hidden/model ratio fixed: a much smaller model (∼0.05B parameters) and a larger one (∼1.82B, the largest this GPU's 17.2 GB comfortably supports with two full activation caches held in memory), correctness invariants re-verified at each before any timing number.

Table 8: Amortization gain vs. model width, three scales, clock locked to 1402 MHz, two independent launches each (range shown; ∼0.81B row reproduced from Table 7), consumers-index fix in place.

Scale (parameters)	Naive gain	Batched gain
∼0.05B (dim 512)	18.5–26.9%	36.3–40.3%
∼0.81B (dim 2048)	25.8–26.6%	37.1–38.9%
∼1.82B (dim 3072)	30.4–31.5%	**43.6–45.7**%

With the confound removed, naive restoration now rises cleanly across all three widths (∼22.7%, ∼26.2%, ∼31.0% midpoints), matching the roofline prediction's direction without exception – a materially cleaner fit than the pre-fix data, which had shown a spurious dip at the middle scale that this section's diagnosis attributes to the same confound expressing itself differently across configurations. The batched kernel's gain exceeds the 36.1% CPU figure at every one of the three widths, with the margin growing at the largest width tested. (Reaching the largest point required one further fix in the same spirit: an initial run exhausted the GPU's available VRAM mid-sweep, producing a 7× timing range and an implausible ordering; garbage-collecting after every repetition rather than every fifth resolved it.) The tested range remains modest (a factor of ∼36 between smallest and largest model), too narrow on this hardware to see whether the trend saturates further.

What this section establishes. Correctness is scale-invariant at all three widths, as the mechanism's device-agnostic design predicts. The amortization gain is not, but its departure from the CPU figure reversed twice over this investigation, both reversals traced to a confound rather than to anything specific to GPU hardware. Two independently *agreeing* measurements were not sufficient evidence against a variable that could bias every run the same way, and a theoretically coherent explanation was not sufficient evidence that the measurement it explained was clean – the same discipline §7.1 applies to software invariants, extended here to hardware-dependent performance claims.

7.8 Validating Circuit Discovery on the Induction Task

Everything above demonstrates the mechanism is fast and correct on a randomly-initialized model – a valid systems benchmark, since recomputation cost depends only on graph structure, but one that establishes nothing about whether the search finds anything meaningful when there is something to find. We train the same architecture on the induction task [9] (a random prefix

of length P, repeated once; predicting the second half's continuation is solvable only by copying from the earlier occurrence) and ask whether the search, unmodified, recovers its documented circuit. We add a token-embedding layer (new, wired on an existing low-level op) and train with the AdamW primitives already in the codebase (3 layers, hidden dimension 64, 4 heads, vocabulary 20, prefix length 8), resampling a fresh prefix every step.

On 100 held-out sequences: 100.0% next-token accuracy on the repeated half versus 16.4% on the unpredictable first half (chance 5%) – the signature of a learned algorithm, verified before any causal analysis. We corrupt the token the induction mechanism must copy from (i_{src}) and probe the prediction at the position that must copy it (j), slicing the recovery metric to row j so generic downstream drift does not mix with the induction-specific effect. Patching the *source* position recovers almost nothing at any layer (≤ 0.001), while patching the *target* position recovers 98.5% already at layer 1 (Table 9) – the corrected information only becomes causally load-bearing once attention has moved it to where it is read.

Table 9: Layer-level recovery under the induction task, source-position vs. target-position patching.

Layer	**Recovery (patch source i_{src})**	**Recovery (patch target j)**
1	+0.0005	0.9854
2	+0.0010	0.9951
3	+0.0000	1.0000

The greedy search, unmodified except for the row-sliced recovery above, searches all 12 heads (3 layers $\times$ 4) with no positional hint. Table 10 shows two layer-1 heads already reaching 99.6% cumulative recovery; three more bring it to 99.94%, matching an independent joint-patch recomputation exactly (Invariant 5).

Table 10: Greedy search trajectory on the trained induction model, 12 candidate heads (3 layers).

Step	**Site added**	**Cumulative recovery**
1	Layer 1, Head 4	0.6499
2	Layer 1, Head 2	0.9961
3	Layer 1, Head 3	0.9987
4	Layer 1, Head 1	0.9992
5	Layer 2, Head 1	0.9994

7.8.1 Independent Confirmation and Backward Pruning

As a check entirely independent of the recovery metric, we inspect each selected head's post-softmax attention at query position j: the two heads found first, jointly responsible for 99.6% of the recovered effect, attend 99.8% and 100.0% of their weight directly to i_{src} – the textbook induction-head signature [9], confirmed by a measurement the causal search never examined.

The search only ever adds sites; it never asks whether one has become redundant once the others are present. One of the five selected sites (Layer 1, Head 1) does not attend to the source position at all, hinting that its retention may reflect a small residual gain rather than genuine participation. A backward-pruning pass (new) tests this directly, but cannot simply undo a site in place: three of the five selected sites lie strictly upstream of the fifth via the residual stream, so removing an upstream site while a downstream one remains active would silently invalidate and discard the downstream patch at the next `demand!` call. Instead, each candidate subset is tested by resetting the full union of all five sites' cones to the corrupted baseline (cache-replay restoration, exact per Invariant 3) and reapplying only that subset in its original order – always

topologically safe, since layer-1 sites already precede the layer-2 site in the search's own order. Applied here, it removes Layer 2, Head 1 – the last-added, only layer-2 site – with no measurable loss (sliced-row recovery 0.9994 → 0.9992; full-output recovery 0.9913 → 0.9919), consistent with its own attention weight on i_{src} (0.121) sitting an order of magnitude below the two dominant layer-1 heads (0.998–1.0). The four remaining layer-1 heads alone constitute the circuit's minimal causal core.

7.9 Extending the Mechanism to Structural Surgery

Everything above treats the graph's structure as fixed. The same persistence that makes a patch cheap – individually addressable nodes, explicit validity state – also makes *mutating* structure mid-training cheap, inheriting the same cone bound rather than requiring a rebuild from scratch. A structural-mutation primitive inserts a new transformer block after a chosen residual-stream node, zero-initializing its output projections so the graft computes the identity function exactly at insertion – verified bit-for-bit on a real trained model. A second construction offers a gated variant: a learnable scalar gate multiplying the branch's output, with the branch's own weights left randomly initialized (a ReZero-style variant).

7.9.1 A Grafted Layer Earns Its Capacity

We insert a fifth layer into a 4-layer character-level language model mid-training, after 10,000 steps on TinyShakespeare, and continue both the grafted model and a fresh 4-layer control for the same total budget (40,000 steps) – a fairer test than grafted-vs-itself, since it asks whether the added capacity is worth its share of the *same* compute.

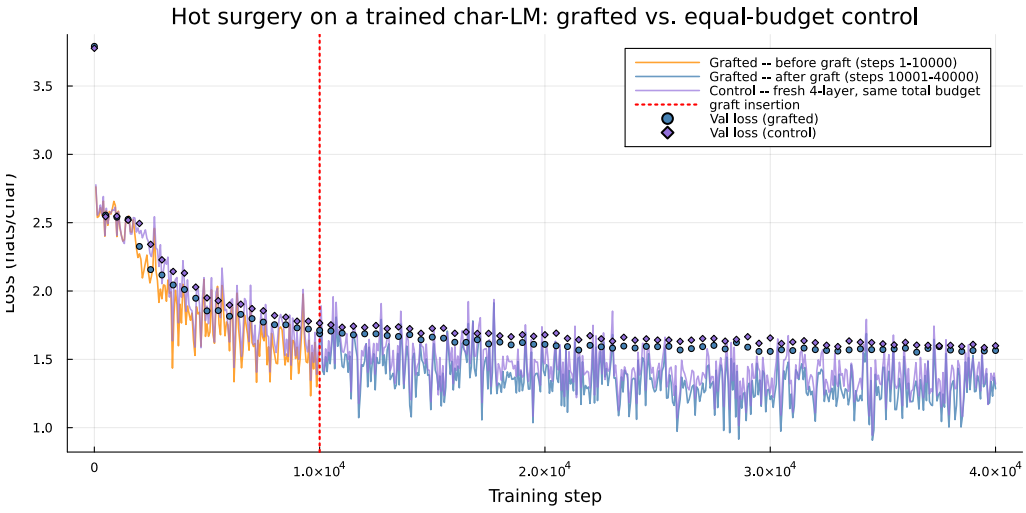

Figure 7: Grafted model (orange before, blue after insertion at step 10,000) vs. a fresh 4-layer control trained for the same total budget. The graft's validation loss ends lower despite starting from a smaller model.

The grafted model reaches a lower final validation loss than the control (1.5647 vs. 1.5995 nats/char) at identical total step count. This is a single run with no seed variation tested; §7.9.2 immediately below runs a closely related grafting comparison over three seeds and finds that a comparison of exactly this kind can invert sign on one of three seeds, so the result above should be read as an observation, not a settled comparison. A greedy circuit search (§5.4) over six frozen test windows – selected to avoid a ceiling effect an earlier attempt hit, where windows already near-saturated at 0.95–1.0 recovery left nothing for added capacity to explain – finds the grafted layer recruited into the retained circuit on one of six windows, with its per-head sweep

on the rest indistinguishable from noise below a pre-registered threshold. The graft's weights leave the identity manifold with substantial, smoothly growing norms, so the added capacity is not dormant: it earns the loss improvement above without acquiring an individually detectable role in this specific circuit.

7.9.2 Does Causal Diagnosis Direct Beneficial Placement? A Negative Result

The natural next question is whether a causal diagnosis can *direct* where new capacity should be grafted, not merely describe the network afterward. We test this directly. A synthetic N-hop associative-recall task is built (randomized-order key-value chain; the correct answer requires traversing the whole chain, so a single-hop induction head answers wrong) specifically because the induction task above saturates too fast for this purpose (full-circuit recovery never dropped below 0.85 across a wide screen of candidate windows). On a 6-layer model trained on the 3-hop variant, per-layer patch recovery varies genuinely across depth-matched layers 2–6 (0.13–0.26, non-monotone with depth, excluding layer 1's pure proximity artifact); layer 3 (recovery 0.26) is designated the *bottleneck*, layer 2 (recovery 0.13) the depth-matched *control*.

We graft the gated construction after each site, same random seed for both initial weights and batch order, so placement is the only difference between arms. Three criteria are pre-registered before training, checked after 9,000 steps over three seeds: (i) the bottleneck-placed graft's gate should exceed the control-placed graft's by more than 2×, stably across seeds; (ii) the bottleneck graft should rank in the top three sites of a post-hoc circuit search, the control outside the top ten; (iii) the bottleneck-placed graft should reach lower loss than the control-placed graft at equal budget.

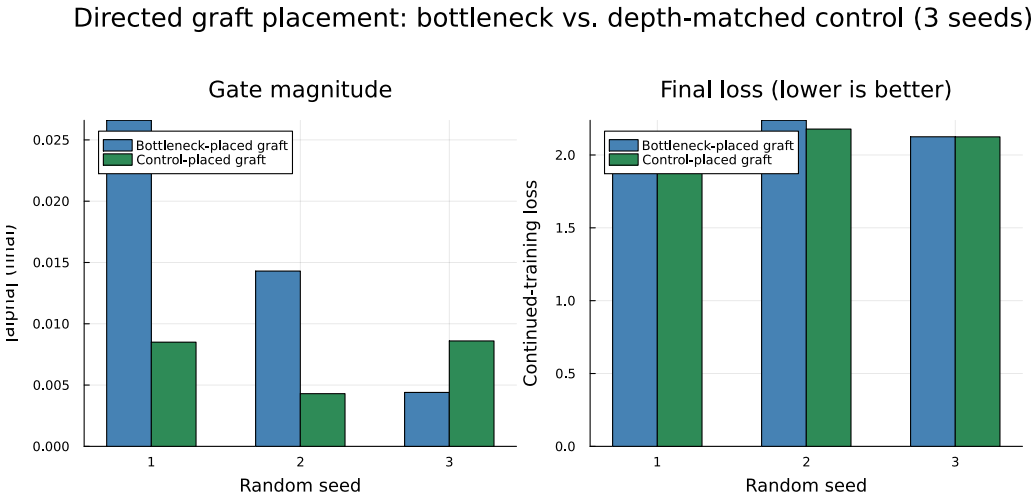

Figure 8: Bottleneck- vs. control-placed graft, three seeds. Neither gate magnitude nor final loss favours the bottleneck placement consistently.

All three criteria fail. (i) holds on two of three seeds but inverts on the third (0.0086 vs. 0.0044, control larger). (ii) fails on every one of six runs: the graft never appears in the top three at either placement, on any seed – the same layer-1 heads are recovered regardless of where it sits. (iii) fails outright: the control-placed graft reaches *lower* loss on two of three seeds (2.0590 vs. 2.1053; 2.1779 vs. 2.2382), the opposite of the hypothesis. We report this as a genuine negative result on this specific setup – this task, this depth, this gate mechanism, nine thousand steps – not evidence that causal diagnosis is never actionable for placement. What §7.9.1–§7.9.2 together establish is that the persistent graph makes the full diagnose-graft-rediagnose cycle cheap enough to test this question empirically, in minutes, rather than leaving it unasked.

8 Discussion and Limitations

From a proven ceiling to a testable claim. Theorem 2 rules out exhaustive sweep coverage as a route to closing the ACDC/AtP* fidelity-cost gap at any scale or cost-by-depth profile, and identifies exactly where the mechanism does help instead: verifying a small number of deep, late-ranked candidates, the second half of AtP*'s own recommended workflow. This paper does not yet run that workflow end to end – ranking candidates with an AtP-style backward pass, then verifying only the top-k with the sweep procedure – on a task with a documented ground-truth circuit; Section 7.8 shows the search machinery finds such a circuit unassisted, and combining that with an AtP-style pre-ranking step is the natural next empirical step.

Scale and task grounding, not yet combined. The 8-layer, dim-128 CPU model used in §7.2–§7.6 isolates the systems mechanism at low iteration cost; Section 7.8 grounds correctness on a real task but stays small and CPU-only, without re-benchmarking the amortized sweep there; Section 7.7 grounds scale but uses random weights, with no circuit to discover; Section 7.9 grounds task on a real trained model but stays small and does not re-benchmark the sweep either. Automated circuit search, amortized for speed, at GPU scale, on a real trained language model with a documented circuit – combining all three dimensions this paper tests only pairwise – remains the natural next step, on a bigger model or a more complex circuit such as IOI.

Limitations. The GPU-scale trend (Table 8) spans a factor of only $\sim$36 between smallest and largest model tested, too narrow on this hardware to see whether it saturates at larger scale. Backward pruning is run once, after the search completes; whether interleaving pruning with addition changes the trajectory is unexplored, since the two-pass version already answers the necessity question this paper asks. §7.9.2's negative result is specific to one task, depth, and gate mechanism, and should not be read more broadly than that. §7.9.1's hot-surgery comparison is a single run with no seed variation, unlike §7.9.2's test immediately following it, which found that exactly this kind of comparison can be seed-sensitive on a closely related task; the loss figures reported there should be read as a single-run observation, not a robust effect size.

9 Conclusion

A causal-tracing sweep is not one patch but many, run against a shared corrupted baseline. Because a single patch's cost is a purely structural quantity, we asked – and answered with a theorem, before any implementation – whether an exhaustive, per-node sweep's aggregate cost converges to a ceiling as depth grows: bounded by an exact pair-counting identity holding at every finite depth, for *every* cost-by-depth profile, with no asymptotic hypothesis, it does – converging to *exactly* 2$\times$ whenever no single layer asymptotically dominates the network's cost (Theorem 2) – strengthening a companion study's depth-uniform-only result [3], and identifying that same hypothesis's failure (a single dominating layer) as what raises the ratio instead to an exactly computable 8/3 (Remark 2). This closes off, on strictly more general grounds than a depth-uniform architecture alone, a real methodological hope: that NeuroDSL's per-site locality might let an exhaustive, ACDC-style sweep scale into a genuine alternative to fast-but-approximate attribution such as AtP*. It cannot, at any scale or profile short of a single dominating layer – but the same closed form shows the mechanism instead makes AtP*'s own recommended top-k exact-verification step, applied to deep and late candidates, nearly free.

We confirmed both halves of this claim empirically. Cache-replay restoration and the sweep procedure realize the saving in practice by recognizing that a sweep's restoration step need not be computed at all, since the corrupted baseline is already known, removing 36.1% of total sweep cost on an 8-layer CPU model, verified correct by exact-equality invariants before any timing number. At $\sim$0.81–1.82-billion-parameter GPU scale, reaching a trustworthy number required diagnosing

and removing two independent confounds – an unlocked GPU clock and an $\mathcal{O}(|\text{cone}| \times |\text{rules}|)$ invalidation-bookkeeping cost – and we report plainly that doing so reversed, rather than merely revised, a roofline-style explanation we had initially offered for the shortfall it left uncorrected: a theoretically coherent explanation for a measurement is not evidence the measurement itself was clean. Cross-validated against a real PyTorch run on identical, exported weights, the single-patch advantage is the depth-dependent crossover the theorem's per-site analysis predicts, trailing at the shallowest site and reaching 10× at the deepest.

Trained on the induction task, the same unmodified search finds the right answer, not just a fast one – a small set of attention heads reaching 99.94% of the causal effect, independently confirmed by their attention patterns and reduced by backward pruning to a four-head minimal core. Extending the same persistence from patching values to mutating structure, a layer grafted into a real trained model outperforms a fresh, equal-budget control, while three pre-registered criteria for letting causal diagnosis *direct* such a graft's placement all fail – a negative result we report as such, because the persistent graph's real contribution here is making the question cheap enough to ask empirically, in minutes, rather than settling for an untested assumption in either direction. A companion paper reports a third use of the same exact-graft primitive – scheduling growth events during training – separately, since it shares only the mechanism with this paper, not its method or audience.

NeuroDSL is open-source at `https://github.com/nevermind78/NeuroDSL`.

References

[1] Khemais, A. (2026). NeuroDSL: A Reactive Computational Graph Framework for Deep Learning in Julia. *Preprint.* Reference implementation: `https://github.com/nevermind78/NeuroDSL`.

[2] Khemais, A. (2026). Exact Network Surgery: Functional Invariance and Gradient Plasticity in Reactive Computational Graphs. *arXiv preprint arXiv:2607.16568.*

[3] Khemais, A. (2026). Cost Accounting for Reactive Computational Graphs: Exhaustive Sweeps, Sequential Mutation, and the Backward-Locality Gap. *arXiv preprint arXiv:2607.18323.*

[4] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS.*

[5] Bradbury, J., et al. (2018). JAX: composable transformations of Python+NumPy programs. *GitHub.*

[6] Meng, K., Bau, D., Andonian, A., Belinkov, Y. (2022). Locating and Editing Factual Associations in GPT. *NeurIPS.*

[7] Vig, J., Gehrmann, S., Belinkov, Y., Qian, S., Nevo, D., Singer, Y., Shieber, S. (2020). Investigating Gender Bias in Language Models Using Causal Mediation Analysis. *NeurIPS.*

[8] Nanda, N., Bloom, J. (2022). TransformerLens. *GitHub*, `https://github.com/TransformerLensOrg/TransformerLens`.

[9] Olsson, C., Elhage, N., Nanda, N., Joseph, N., DasSarma, N., Henighan, T., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Johnston, S., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., Olah, C. (2022). In-context Learning and Induction Heads. *Transformer Circuits Thread.*

[10] Nemhauser, G. L., Wolsey, L. A., Fisher, M. L. (1978). An Analysis of Approximations for Maximizing Submodular Set Functions – I. *Mathematical Programming* 14(1).

[11] Williams, S., Waterman, A., Patterson, D. (2009). Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM* 52(4).

[12] Conmy, A., Mavor-Parker, A. N., Lynch, A., Heimersheim, S., Garriga-Alonso, A. (2023). Towards Automated Circuit Discovery for Mechanistic Interpretability. *NeurIPS*.

[13] Kramár, J., Lieberum, T., Shah, R., Nanda, N. (2024). AtP*: An Efficient and Scalable Method for Localizing LLM Behaviour to Components. *arXiv preprint arXiv:2403.00745*.